期中速查表

语法 · 类型 · 列表 · 结构递归

范围：目录第 1-13 讲 + 第 14、15 讲 PDF；不展开范畴论附录。各模板独立使用，等号亦表示等价关系。

01 语法与类型推导

第 3-5 讲

应用优先级最高且左结合；箭头右结合。

```
f x y = (f x) y
a -> b -> c = a -> (b -> c)
(f . g) x = f (g x)   -- 先 g 后 f
f $ x = f x           -- $ 右结合、优先级 0
(x +) = \y -> x + y
(+ y) = \x -> x + y
(-2)                  -- 负数；减 2 用 subtract 2
```

常用优先级（高到低；L/R 表示左/右结合）：

```
9R .     9L !!     8R ^,^^,**
7L *,/,`div`,`mod`   6L +,-   5R :,++
4N ==,/=,<,<=,>,>=  4L <$>,<*>,<*,*>
3R &&   3L <|>   2R ||   1L >>=,>>   0R $,$!
```

N：不可连写；比较链写 x < y && y < z。

```
f x | p x       = a
    | otherwise = b
g xs = case xs of { [] -> z; x:ys -> h x ys }
let x = e in body     -- do 内 let 无 in
```

模式从上到下匹配；变量不可在同一模式重复；(x:xs) 匹配非空表，[x] 恰好一个元素。

02 约束与数值转换

第 4、9 讲

```
(==),(/=) :: Eq a => a -> a -> Bool
compare :: Ord a => a -> a -> Ordering
(+),(-),(*) :: Num a => a -> a -> a
(/) :: Fractional a => a -> a -> a
div,mod :: Integral a => a -> a -> a
fromIntegral :: (Integral a, Num b) => a -> b
show :: Show a => a -> String
read :: Read a => String -> a
```

Ord 要求 Eq；Integral 与 Fractional 均属于 Num 体系。Int 定长，Integer 任意精度；String = [Char]。

```
fromIntegral (length xs) / 2
read "123" :: Int
(-5) `div` 2 = -3; (-5) `mod` 2 = 1
```

推导类型：为形参设类型变量，逐次应用统一输入/输出类型，再合并 Eq/Num 等约束。

```
(\f x -> f (f x)) :: (a -> a) -> a -> a
(\x y -> x == y) :: Eq a => a -> a -> Bool
```

03 列表与常用积木

第 5-8 讲

```
(:)  :: a -> [a] -> [a]
(++) :: [a] -> [a] -> [a]
map :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a,b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
concat :: [[a]] -> [a]
concatMap :: (a -> [b]) -> [a] -> [b]
take,drop :: Int -> [a] -> [a]
takeWhile,dropWhile :: (a -> Bool) -> [a] -> [a]
```

zip 按较短表截断；takeWhile 遇首个不满足者停止，filter 检查所有元素。head/tail/last/init 对空表出错；!! 下标从 0 起且不能越界。

```
[f x y | x <- xs, y <- ys, p x y]
-- x 外层，y 内层；可依赖前面的 x
[(x,i) | (x,i) <- zip xs [0..], p x]
```

04 Semigroup / Monoid

第 14 讲

```
(<>) :: Semigroup a => a -> a -> a
mempty :: Monoid a => a
mconcat = foldr (<>) mempty
```

<> 须结合；Monoid 要求 mempty 为左右单位元。[]: ++ / []；Sum: + / 0；Product: * / 1；All: && / True；Any: || / False。包装类型从 Data.Monoid 导入。

05 Fold：最值得查的展开式

第 3、8 讲

以下先写列表专用签名；标准库推广为 Foldable t 时，把 [a] 换成 t a 并加约束。

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []     = z
foldr f z (x:xs) = f x (foldr f z xs)
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z []     = z
foldl f z (x:xs) = foldl f (f z x) xs
foldr f z [a,b,c] = f a (f b (f c z))
foldl f z [a,b,c] = f (f (f z a) b) c
foldr (-) 0 [1,2,3] = 2
foldl (-) 0 [1,2,3] = -6
```

折叠方向决定括号，f 无需满足结合律。foldl 也惰性；严格累积用 Data.List.foldl'。

```
sum     = foldr (+) 0
product = foldr (*) 1
and     = foldr (&&) True
or      = foldr (||) False
length  = foldr (\_ n -> 1+n) 0
map g   = foldr (\x ys -> g x : ys) []
filter p = foldr (\x ys ->
                  if p x then x:ys else ys) []
reverse = foldl (flip (:)) []
xs ++ ys = foldr (:) ys xs
-- 自然数折叠 (n >= 0；课程自定义)
foldN h z 0 = z
foldN h z n = h (foldN h z (n-1))
```

06 数据类型与结构递归

第 9、12、14 讲

```
type Pair a = (a,a)          -- 别名
newtype Box a = Box a        -- 新类型，单字段
data Tree a = Leaf a | Node (Tree a) (Tree a)
              deriving (Eq, Show)
Leaf :: a -> Tree a
Node :: Tree a -> Tree a -> Tree a
```

Tree 是类型构造器；Leaf/Node 是值构造器。newtype 与原类型不自动互换，也不自动检查取值范围。

```
instance Functor Tree where
  fmap f (Leaf x)   = Leaf (f x)
  fmap f (Node l r) = Node (fmap f l) (fmap f r)
instance Foldable Tree where
  foldMap f (Leaf x) = f x
  foldMap f (Node l r) = foldMap f l <> foldMap f r
instance Traversable Tree where
  traverse f (Leaf x) = Leaf <$> f x
  traverse f (Node l r) =
    Node <$> traverse f l <*> traverse f r
```

通用结构递归：每个构造器一条分支；递归字段先递归，再组合。自定义 fold 就是替换构造器。

```
data Expr = Val Int | Add Expr Expr
folde v a (Val n)   = v n
folde v a (Add l r) = a (folde v a l) (folde v a r)
eval = folde id (+)
```

07 Countdown 的搜索骨架

第 10 讲

```
subs [] = [[]]
subs (x:xs) = let ys = subs xs in ys ++ map (x:) ys
perms [] = [[]]
perms (x:xs) = concatMap (interleave x) (perms xs)
choices = concatMap perms . subs
```

subs 是保序子序列；interleave x xs 枚举插入位置；split 枚举两个非空连续部分。融合生成与求值，携带 (Expr,Int)，尽早排除非法分支。

```
-- 正整数 x,y；剪枝版 valid
Add: x <= y       Sub: x > y
Mul: x <= y && x /= 1 && y /= 1
Div: x `mod` y == 0 && y /= 1
```

期中速查表

计算上下文 · 解析器 · 折叠 · 惰性

范围：目录第 1-13 讲 + 第 14、15 讲 PDF；不展开范畴论附录。各模板独立使用，等号亦表示等价关系。

08 Functor / Applicative / Monad

第 12 讲

能力关系：Monad 要求 Applicative，后者要求 Functor。f/m 表示还差一个类型参数的类型构造器。

```
fmap  :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap
pure  :: Applicative f => a -> f a
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
(>>=) :: Monad m => m a -> (a -> m b) -> m b
(>>)  :: Monad m => m a -> m b -> m b
mx >> my = mx >>= \_ -> my
return = pure
f <$> mx <*> my = pure f <*> mx <*> my
```

fmap 改内部值；<*> 合并上下文；>>= 允许后一步计算依赖前一步的值。

```
-- do 脱糖（这里 x 是变量模式）
do { x <- mx; k x } = mx >>= \x -> k x
do { mx; my }       = mx >> my
-- Functor laws
fmap id = id
fmap (f . g) = fmap f . fmap g
-- Applicative laws
pure id <*> u = u
pure f <*> pure x = pure (f x)
u <*> pure y = pure ($ y) <*> u
pure (.) <*> u <*> v <*> w = u <*> (v <*> w)
-- Monad laws
pure x >>= f = f x
mx >>= pure = mx
(mx >>= f) >>= g = mx >>= (\x -> f x >>= g)
-- Control.Monad 的 Kleisli 组合
(f >=> g) x = f x >>= g
```

09 典型实例：直接套用

第 12 讲

```
-- Maybe：失败传播
fmap f Nothing = Nothing
fmap f (Just x) = Just (f x)
pure x = Just x
Just f <*> mx = fmap f mx
Nothing <*> _ = Nothing
Nothing >>= f = Nothing
Just x  >>= f = f x
-- []：所有组合，保留枚举顺序
pure x = [x]
fs <*> xs = [f x | f <- fs, x <- xs]
xs >>= f = concatMap f xs
-- ((->) r)：共享同一个环境 r
fmap = (.)
pure = const
(f <*> g) r = f r (g r)
(h >>= k) r = k (h r) r
```

StateTrans：app st s = (结果,新状态)，不是反过来。以下 pure 与 >>= 写在对应实例中。

```
type State = Int
newtype StateTrans a = ST (State -> (a,State))
app (ST f) = f
pure x = ST $ \s -> (x,s)
st >>= k = ST $ \s -> let (x,s') = app st s
                      in app (k x) s'
fresh = ST $ \n -> (n,n+1)
```

10 IO 只需这些

第 11-12 讲

```
getChar :: IO Char       getLine :: IO String
putChar :: Char -> IO ()
putStr,putStrLn :: String -> IO ()
print :: Show a => a -> IO ()
main = do
  s <- getLine
  let n = read s :: Int
  print (n+1)
```

IO a 是动作；<- 绑定动作的结果，let 定义纯值。return 不会提前退出，IO a 不能当 a 直接算。

11 Parser 组合模板

第 13 讲

课程 Parser 结果限 0 或 1 个；[] 失败，[(值,余串)] 成功。pure 不消耗输入。

```
newtype Parser a = P (String -> [(a,String)])
app (P f) = f
-- 下列 pure、>>=、<|> 放在各自实例中
pure x = P $ \s -> [(x,s)]
p >>= k = P $ \s -> case app p s of
  []        -> []
  [(x,out)] -> app (k x) out
p <|> q = P $ \s -> case app p s of
  [] -> app q s       -- 从原输入重新尝试
  rs -> rs
empty = P $ \_ -> []
item = P $ \s -> case s of
  [] -> []; (x:xs) -> [(x,xs)]
sat p = do { x <- item;
             if p x then pure x else empty }
char c = sat (== c)
many p = some p <|> pure []    -- 0 次或更多
some p = (:) <$> p <*> many p -- 1 次或更多
nat = read <$> some (sat isDigit)
space = many (sat isSpace) >> pure ()
token p = do { space; x <- p; space; pure x }
symbol s = token (string s)
string [] = pure []
string (x:xs) = (:) <$> char x <*> string xs
```

使用 import Control.Applicative、Data.Char。many 的参数若成功却不消耗输入，可能不终止。

```
expr = do { t <- term;
  (do { symbol "+"; e <- expr; pure (t+e) })
    <|> pure t }
term = do { f <- factor;
  (do { symbol "*"; t <- term; pure (f*t) })
    <|> pure f }
factor = (do { symbol "("; e <- expr;
               symbol ")"; pure e }) <|> token nat
```

此文法右递归，* 高于 +；加上减/除后仍右结合。完整解析须得到 [(v,"")]；有余串不能当作整串成功。

12 Foldable / Traversable

第 14 讲

```
fold :: (Foldable t, Monoid a) => t a -> a
foldMap :: (Foldable t, Monoid b)
        => (a -> b) -> t a -> b
fold = foldMap id
traverse :: (Traversable t, Applicative f)
         => (a -> f b) -> t a -> f (t b)
sequenceA :: (Traversable t, Applicative f)
          => t (f a) -> f (t a)
sequenceA = traverse id
```

fold 从 Data.Foldable 导入；Foldable 最少实现 foldMap 或 foldr。Traversable 要求 Functor + Foldable，保留结构并汇集效果。

```
sequenceA [Just 1,Just 2] = Just [1,2]
sequenceA [Just 1,Nothing] = Nothing
```

13 惰性求值与无限结构

第 15 讲

惰性 = 按需（最外策略）+ 共享。按值先算参数；按名可能重复算参数；共享使同一 thunk 求值后复用。

```
const 1 undefined = 1
take 3 (repeat 1) = [1,1,1]
fibs :: [Integer]
fibs = 0:1:zipWith (+) fibs (tail fibs)
iterate f x = x : iterate f (f x)
```

foldr 若能不求第二参数就产出，可处理无限表；foldl 必须走到表尾。length/reverse/sum 通常不能给出无限表的完整结果。$! 仅强制参数到 WHNF（最外构造器或函数），不深算所有字段。

```
const 1 $! undefined = undefined
const 1 $! [undefined] = 1
```